UNIVERSITÄT
LEIPZIG

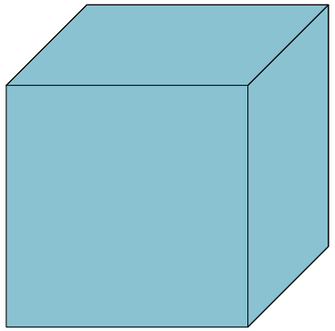
Nebenläufigkeit

Entwicklung verteilter Anwendungen

Leipzig, 03.05.2022

Dr. Pascal Kovacs

AGENDA



Java

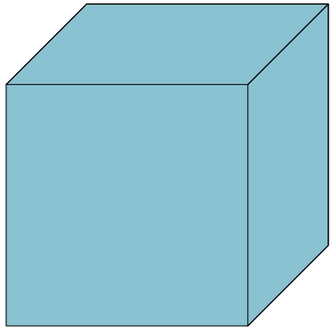
ZIELE JAVA GRUNDLAGEN

- Java als Sprache
- Anweisungen, Operatoren
- Objektorientierung, Vererbung und Interfaces
- Exceptionhandling, Collections, Wrapperklassen

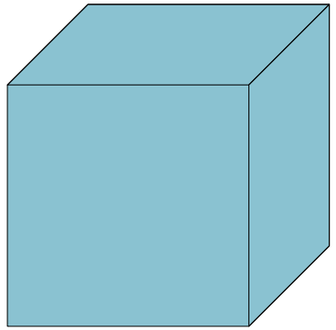
KONTROLLFRAGEN

- Was sind die Designziele der Sprache Java?
- Was ist der Unterschied zwischen primitiven und Referenz-Datentypen?
- Wozu dient der instanceof Operator?
- Wie funktioniert Vererbung in Java?
- Was sind Java Interfaces, welche Bestandteile können sie enthalten?
- Was ist die Klasse Object?
- Wie sind Strings in Java umgesetzt und wie wird mit ihnen gearbeitet?
- Welche zwei Arten von Exceptions gibt es in Java?
- Welche zwei Möglichkeiten der Behandlung von Exceptions gibt es in Java?
- Was ist der Unterschied zwischen Collections und Arrays?

AGENDA



Java



Nebenläufigkeit

ZIELE NEBENLÄUFIGKEIT

- Unterschied serielle/parallele Verarbeitung verstehen
- Motivation & Herausforderungen nebenläufiger Verarbeitung kennen
- Begriffe & Techniken nebenläufiger Systeme kennen
- Probleme nebenläufiger Verarbeitung verstehen

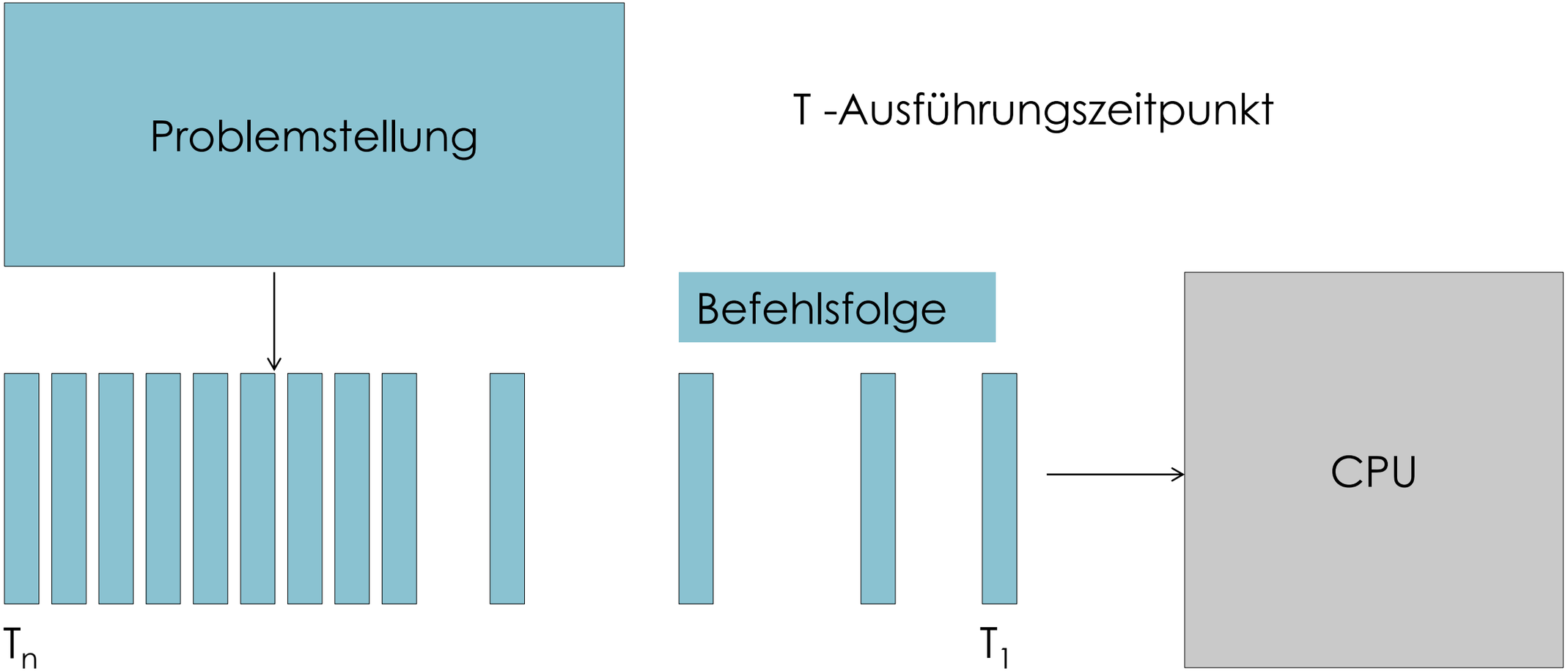
ÜBERSICHT

- Serielle und parallele Verarbeitung
- Kontext der Nebenläufigkeit
- Probleme nebenläufiger Systeme
- Theoretische Modelle zur Nebenläufigkeit

SERIELLE VERARBEITUNG

- Im *Normalfall* wird ein Programm seriell abgearbeitet
- Problemstellung wird in diskrete Abfolge von Instruktionen zerteilt
- Abarbeitung der Befehle erfolgt nacheinander
- Ausführung erfolgt auf einer einzelnen Recheneinheit
- Je Zeiteinheit wird lediglich ein Befehl ausgeführt/abgearbeitet

SERIELLE VERARBEITUNG



BEISPIEL BERECHNUNG PRIMZAHLEN

- Es sollen alle Primzahlen kleiner 1.000.000 ermittelt werden
- Test auf Primzahl erfolgt durch Suche möglicher Teiler pro Zahl
- Ergibt die Teilung den Rest 0 handelt es sich um keine Primzahl
- Weiter mit der nächsten Zahl
- Sind alle möglichen Teiler untersucht worden, ist es eine Primzahl

- Sequentielle Abarbeitung erfolgt durch nacheinander ausgeführte Untersuchung aller ungerader Zahlen, beginnend mit 3

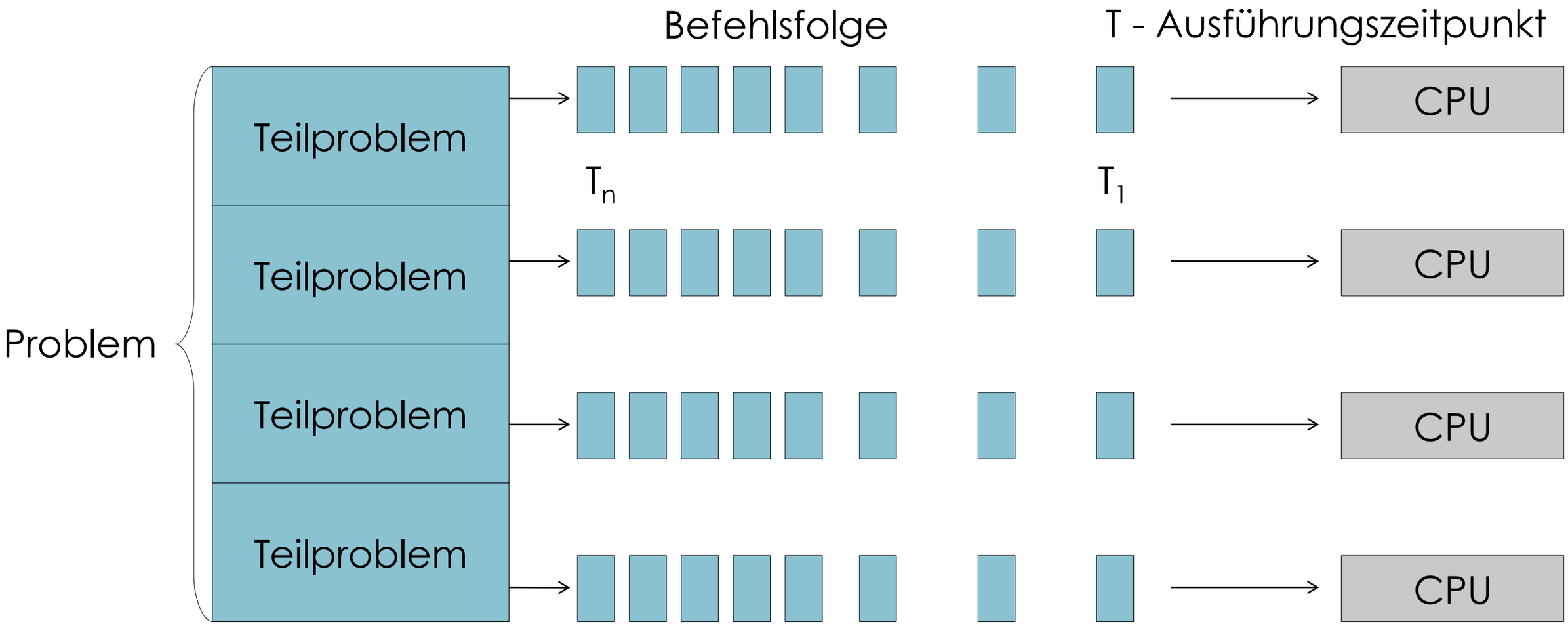
PSEUDOCODE

```
zahl = 3
while ( zahl < 1000000) {
    teiler = 3
    while ( teiler < zahl ) {
       if ( zahl mod teiler == 0 ) {
          break
       } else {
          teiler = teiler + 2
       }
    }
    if ( teiler >= zahl ){
       primzahlen[] = zahl
    }
    zahl = zahl + 2
}
```

PARALLELE VERARBEITUNG

- ...ist: Mehrere Befehle zur gleichen Zeit ausführen
- Erfolgt durch gleichzeitige Nutzung mehrerer Prozessoren/Rechner
- Problem wird in diskrete Teilprobleme zerlegt
- Herunterbrechen der Teilprobleme auf Serie von Instruktionen
- Ausführung auf mehreren Recheneinheiten (Prozessoren/Rechnern)
- Teilprobleme werden somit parallel gelöst
- Das Problem wird parallel gelöst

PARALLEL VERARBEITUNG



BEISPIEL PARALLELE BERECHNUNG VON PRIMZAHLEN

- Mögliche Problemzerlegung zur Parallelisierung:
- Gleicher Untersuchungsablauf
  - Zahlenbereich wird in 4 gleiche Teile a 250.000 Zahlen zerlegt
  - Es werden parallel 4 Zahlen verarbeitet
- Anderer Untersuchungsablauf
  - Aufteilung des Zahlenbereichs der Teiler
  - Prüfung einer Primzahl wird auf 4 parallele Verarbeitungen verteilt
  - Hypothese, dass Nicht-Primzahlen so schneller gefunden werden
  - Greift vielleicht aber erst bei sehr großen Zahlen

PSEUDOCODE

```
Thread1.check_prim(3, 250000,
   primzahlen)
Thread2.check_prim(250000, 500000,
   primzahlen)
Thread3.check_prim(500000, 750000,
   primzahlen)
Thread4.check_prim(750000, 1000000,
   primzahlen)
Wait until all threads finished
```

```
Function check_prim(min, max,
   primzahlen[]):
    while ( min < max) {
        teiler = 3
        while ( teiler < min ) {
            if ( min  mod teiler == 0 ) {
                break
            } else {
                teiler = teiler + 2
            }
        }
        if ( teiler >= min ){
            primzahlen[] = min
        }
        min = min + 2
    }
```

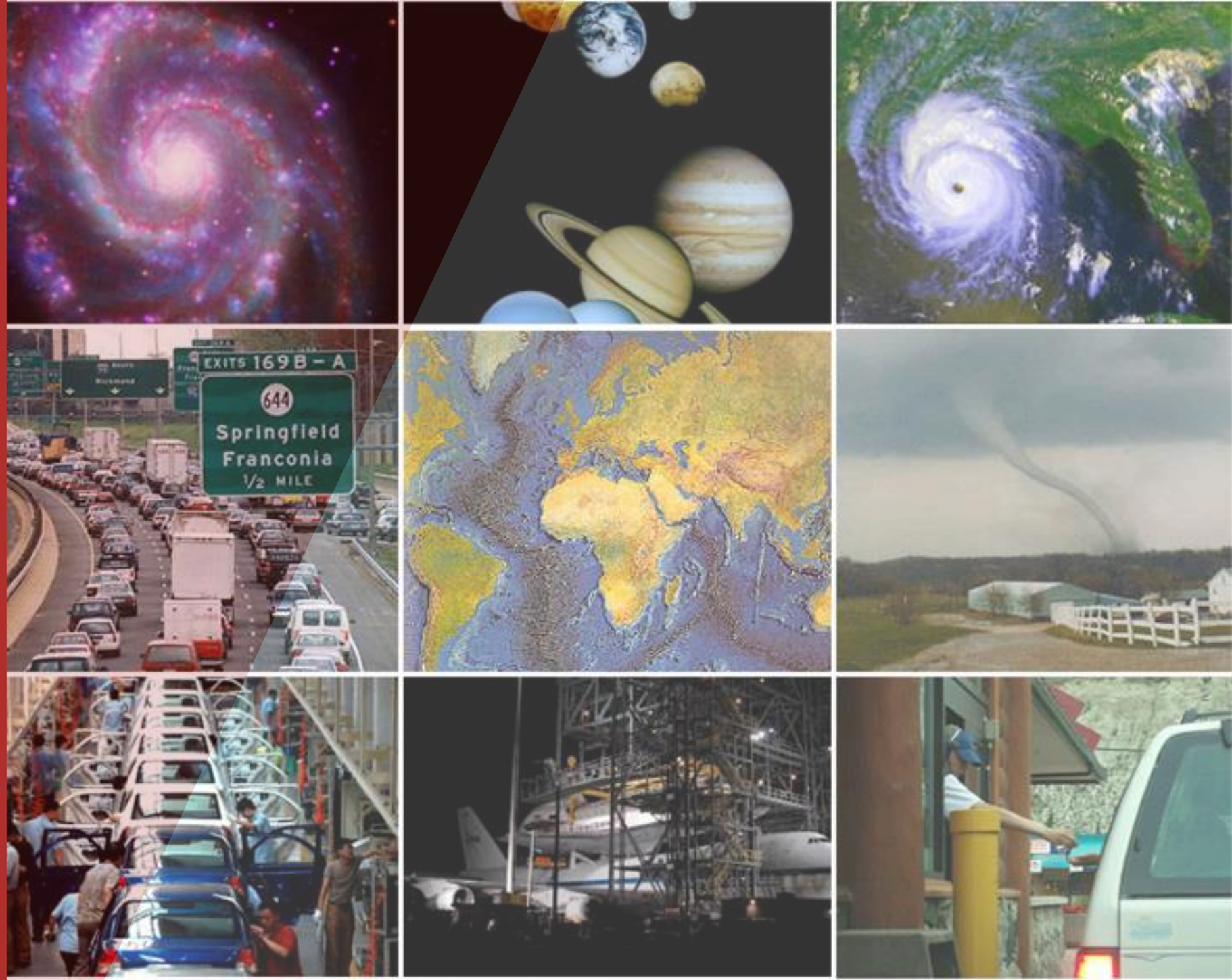
UNTERSCHEIDUNG PARALLELER VERARBEITUNG

- Gleicher Algorithmus mehrfach parallel mit verschiedenen Daten
  - Funktioniert, so lange sich Eingangs- und Ausgangsdaten nicht bedingen
  - Primzahlvariante 1 Aufteilung Zahlenintervall funktioniert
  - Primzahlvariante 2 Aufteilung Teilerintervall funktioniert nicht!
- Identifizieren einzelner Teilaufgaben zur parallelen Verarbeitung
  - Identifikation der einzelnen Teilaufgaben eventuell nicht trivial
  - Koordination der Abhängigkeiten zwischen Aufgaben aufwändig
  - Beispiel eines parallel arbeitenden Systems mit Aufgabentrennung
    - 1) Einlesen und Konvertieren der Daten von einer Datenquelle
    - 2) Verarbeiten der eingelesenen Daten
    - 3) Anzeige der verarbeiteten Daten
    - 4) Export/Logging der verarbeiteten Daten

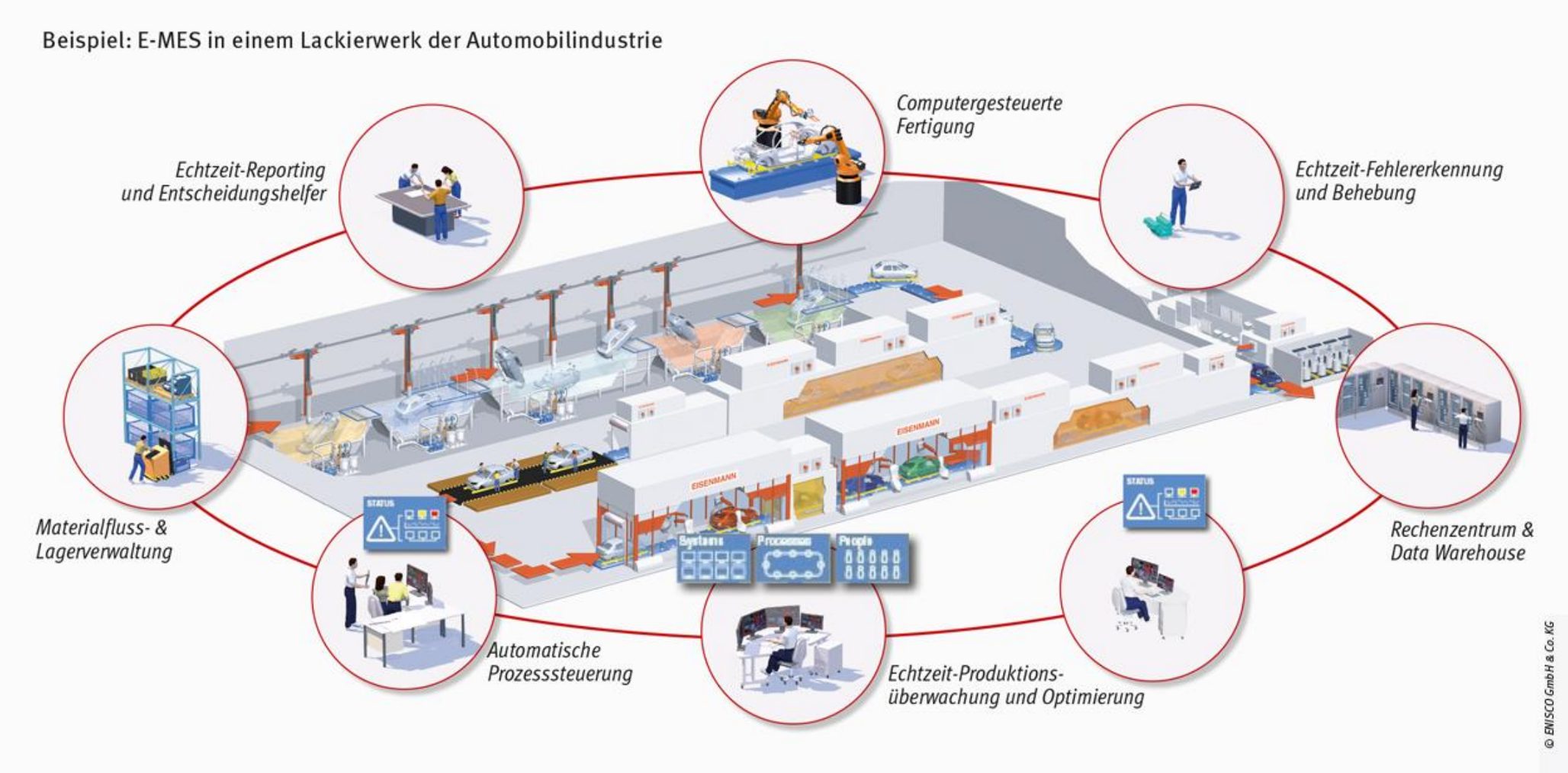
MOTIVATION

- Serielle Verarbeitung...
- Parallele Verarbeitung...
- Aber wozu?

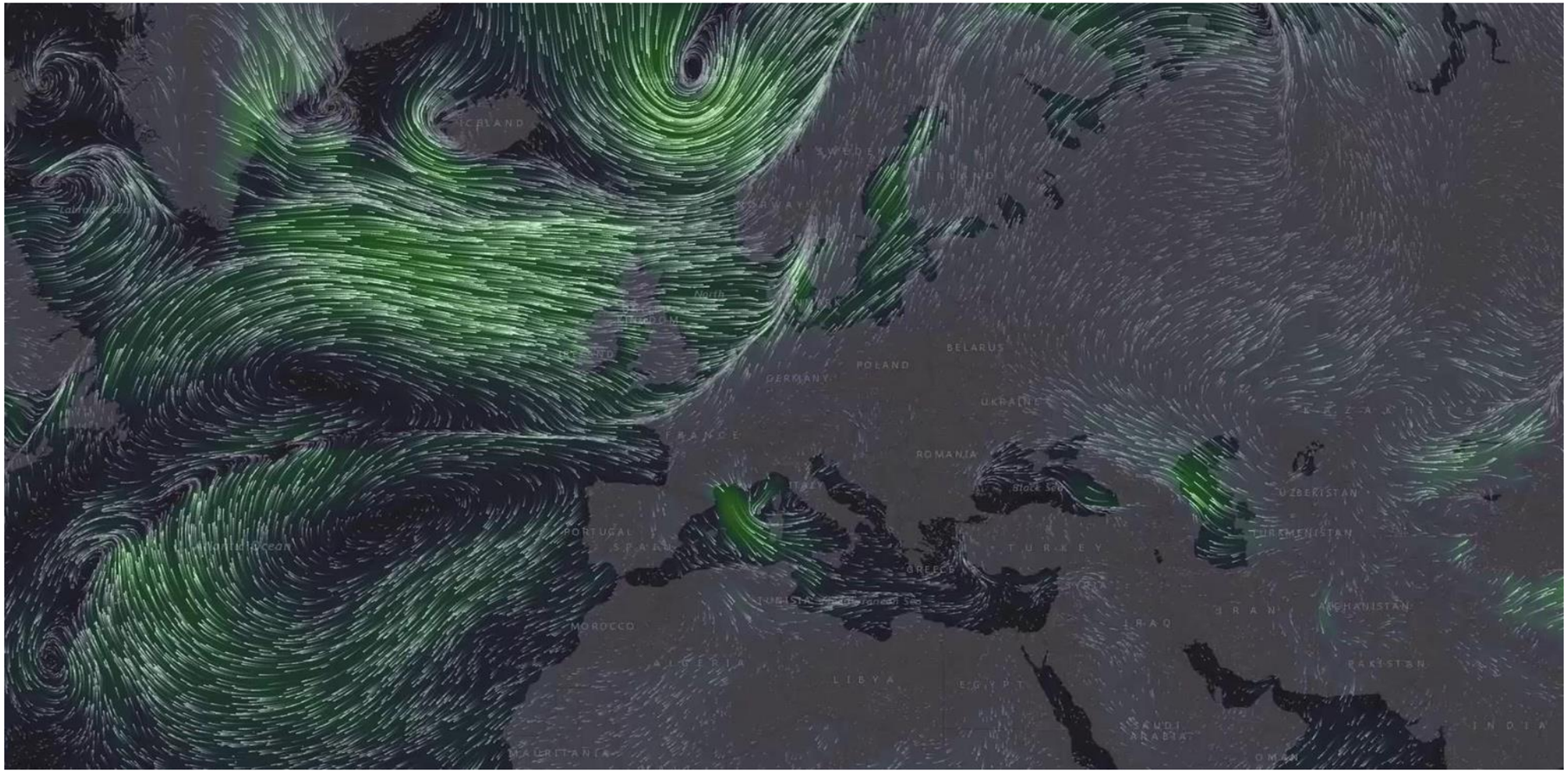
HERAUSFORDERUNGEN DER REALEN WELT AN DIE IT SIND RIESIG!



KOMPLEXE PRODUKTIONSABLÄUFE



PHYSIKALISCHE PROZESSE VERSTEHEN/VORHERSAGEN



ANALYSE GIGANTISCHER DATENMENGEN

KOMPLEXE MÄRKTE ABBILDEN

ANWENDUNGSFÄLLE FÜR PARALLELE VERARBEITUNG

- Umfangreiche Datengrundlage
  - Eine Recheneinheit irgendwann immer überfordert (Physikalische Grenze)
- Verarbeitungsgeschwindigkeit nicht akzeptabel
  - Bearbeitung muss in einer bestimmten Zeit erfolgen
- Verfügbarkeit
  - Ein System muss gegen Ausfall geschützt sein
- Usability
  - Arbeiten mit mehreren Aufgaben gleichzeitig notwendig
- Komplexe Problemstellungen
  - Reale Vorgänge sind oft ein Komplex gleichzeitiger Aktivitäten

ZIELE PARALLELER VERARBEITUNG

- Verarbeitung in endlicher Zeit erreichen
- Beschleunigung einer Verarbeitung
- Höhere Verfügbarkeit des Systems
- Arbeiten an mehreren Aufgaben ermöglichen
- Komplexe Problemstellung handhaben

BEGRIFF DER NEBENLÄUFIGKEIT

- Eigenschaft von Systemen, welche mehrere Verarbeitungen parallel ausführen
- Die verschiedenen Verarbeitungen interagieren oder beeinflussen sich dabei gegebenenfalls gegenseitig
- Systemtheoretisches Modell der parallelen Verarbeitung eines Problems ohne detaillierte Betrachtung der Ausführungsumgebung

KEHRSEITE

- Interaktion paralleler Verarbeitung erhöht Anzahl der Ausführungspfade eines Systems → Das Verständnis leidet
- Gemeinsame Nutzung von Ressourcen kann zu Indeterminismus und spontan fehlerhaftem Verhalten von Programmen führen
- Notwendigkeit von koordinierenden Techniken für Ausführung und Ressourcenzuweisung erhöht Realisierungs- und Wartungsaufwand
- Aufgabenzerteilung und -aufteilung in Teilprobleme ist nicht trivial
- Nebenläufige Programmierung ist deutlich komplexer und schwerer zu realisieren (Programmierkonstrukte, Datenabhängigkeiten)

NEGATIVE FOLGEN

- Mehraufwand für Koordination der Nebenläufigkeit
- Fehler sind schwer zu lokalisieren, da oft nicht deterministisch
- System ist insgesamt schwerer verständlich

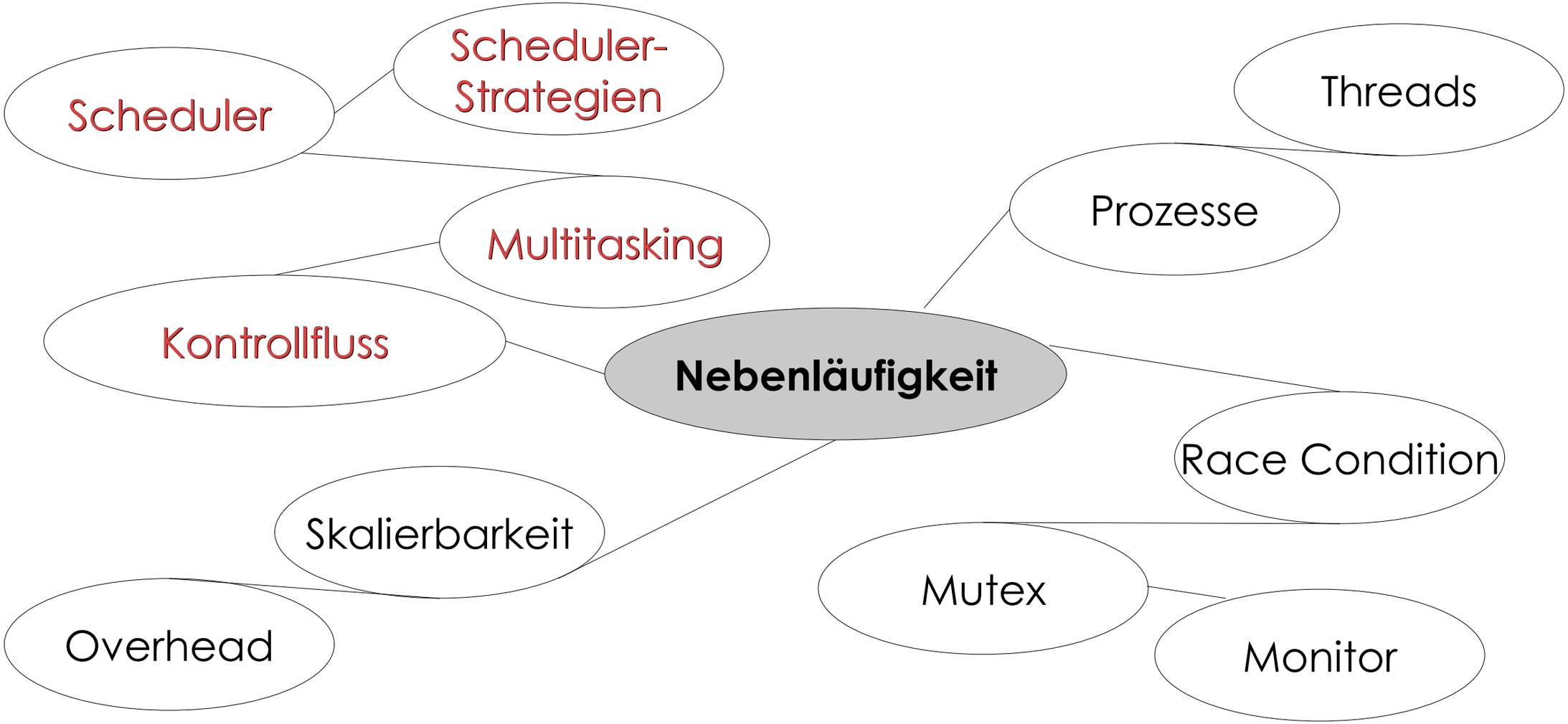
- Höherer Aufwand für programmiertechnische Umsetzung
- Höherer Aufwand für Fehleridentifikation und -Behebung
- Höherer Aufwand bei Service und Wartung

Bei gleichem Umfang und erhöhtem Leistungsdurchsatz (durch Nebenläufigkeit) steigen die Aufwände für die Entwicklung und Qualitätssicherung deutlich an!

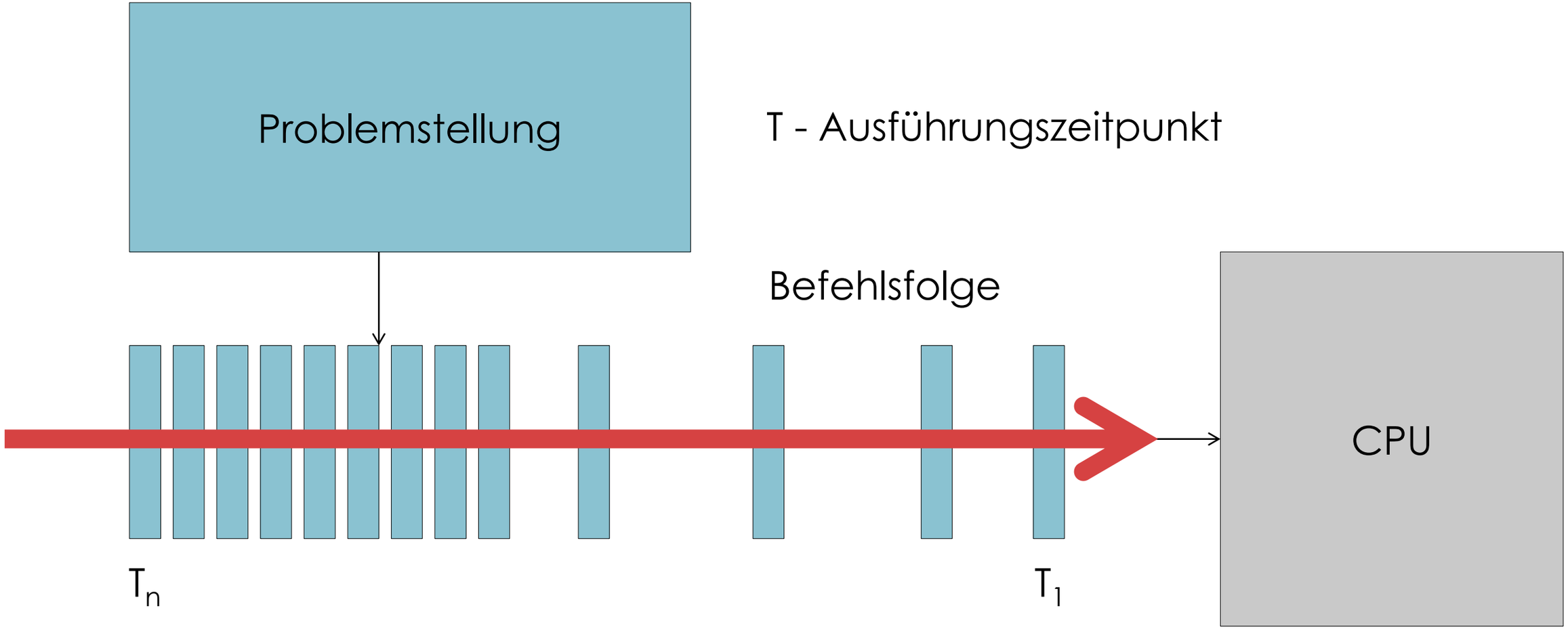
ÜBERSICHT

- Serielle und parallele Verarbeitung
- Kontext der Nebenläufigkeit
- Probleme nebenläufiger Systeme
- Theoretische Modelle zur Nebenläufigkeit

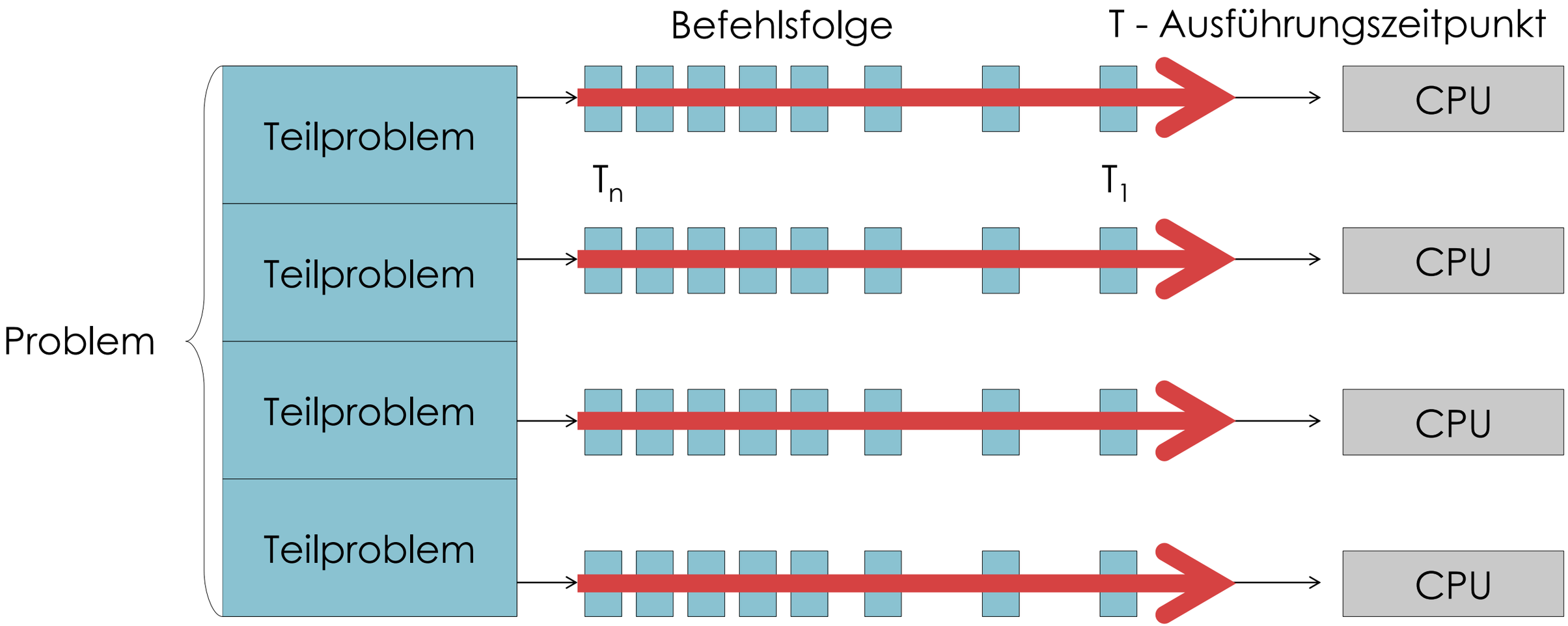
KONTEXT DER NEBENLÄUFIGKEIT



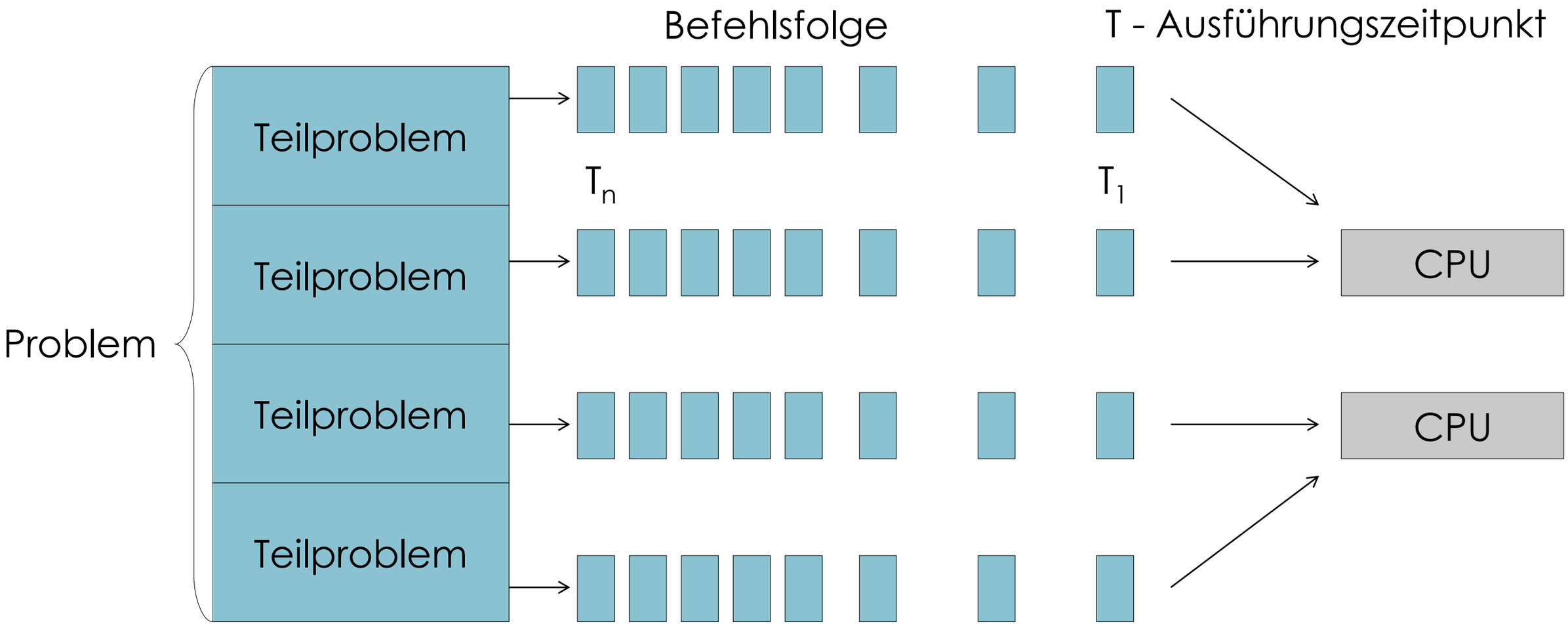
EINZELNER KONTROLLFLUSS



MEHRERE KONTROLLFLÜSSE

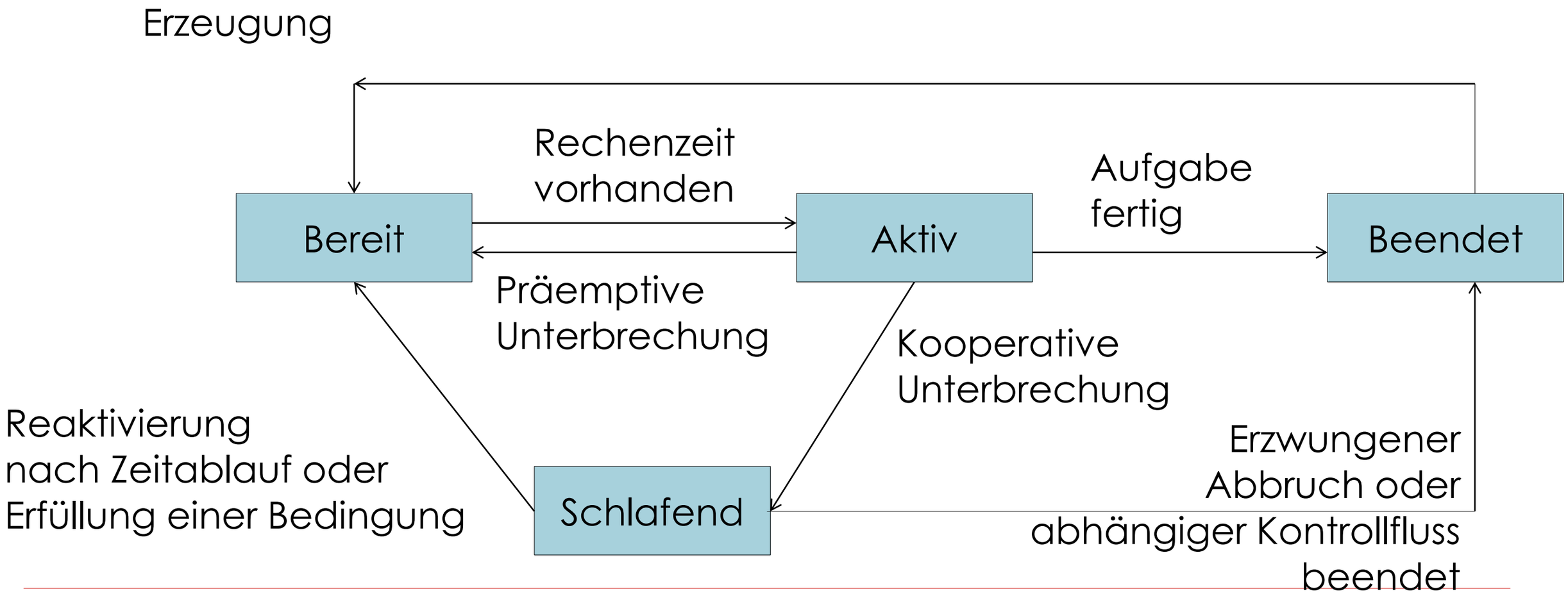


MULTITASKING MIT MEHR KONTROLLFLÜSSE ALS RECHENEINHEITEN

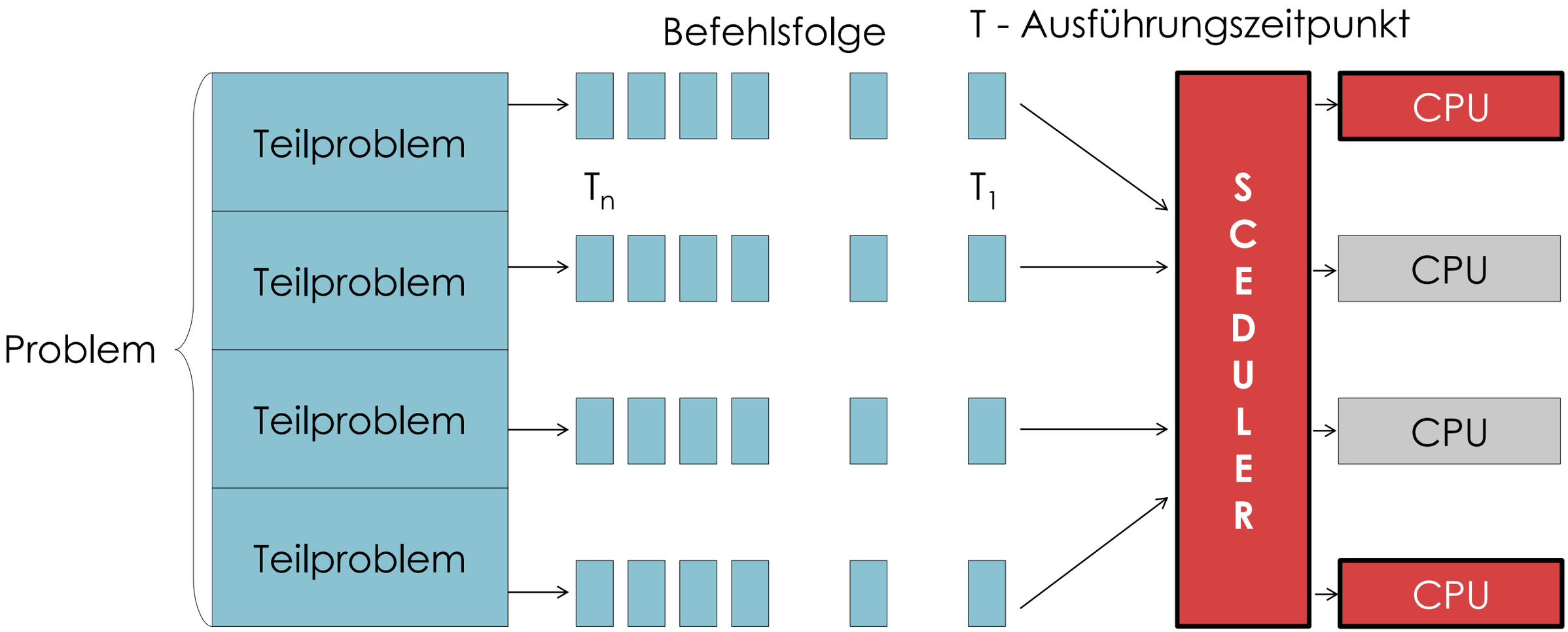


KONTROLLFLUSSLEBENSZYKLUS

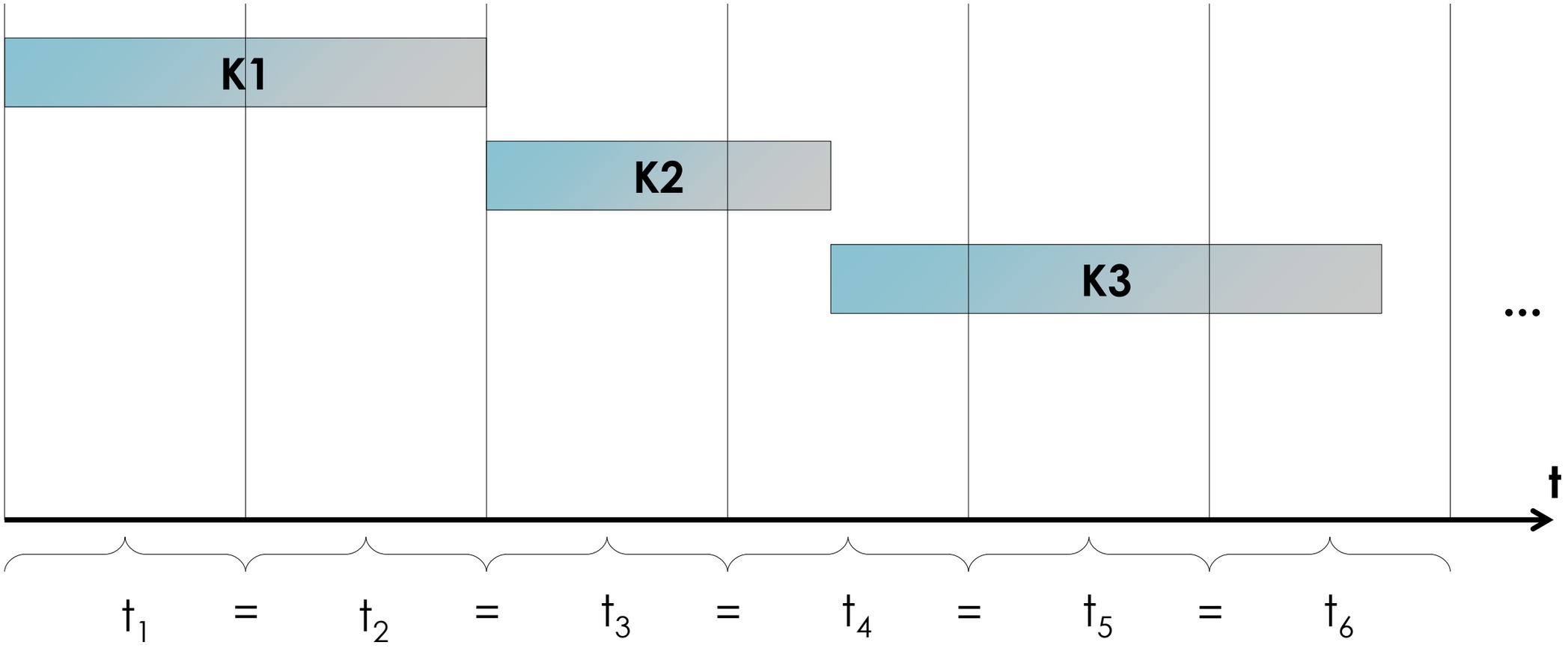
(Kontrollfluss == Berechnungsstrom)



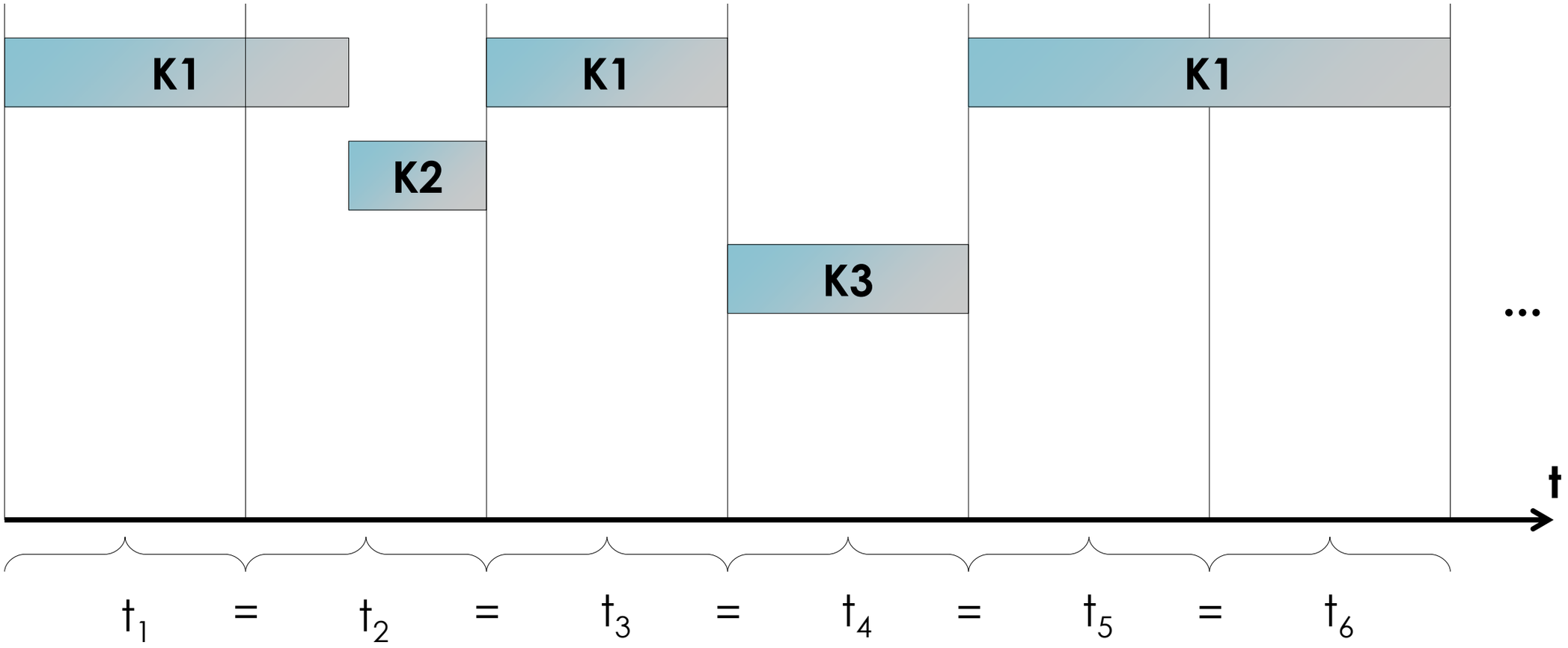
SCHEDULER ALS VERTEILERINSTANZ



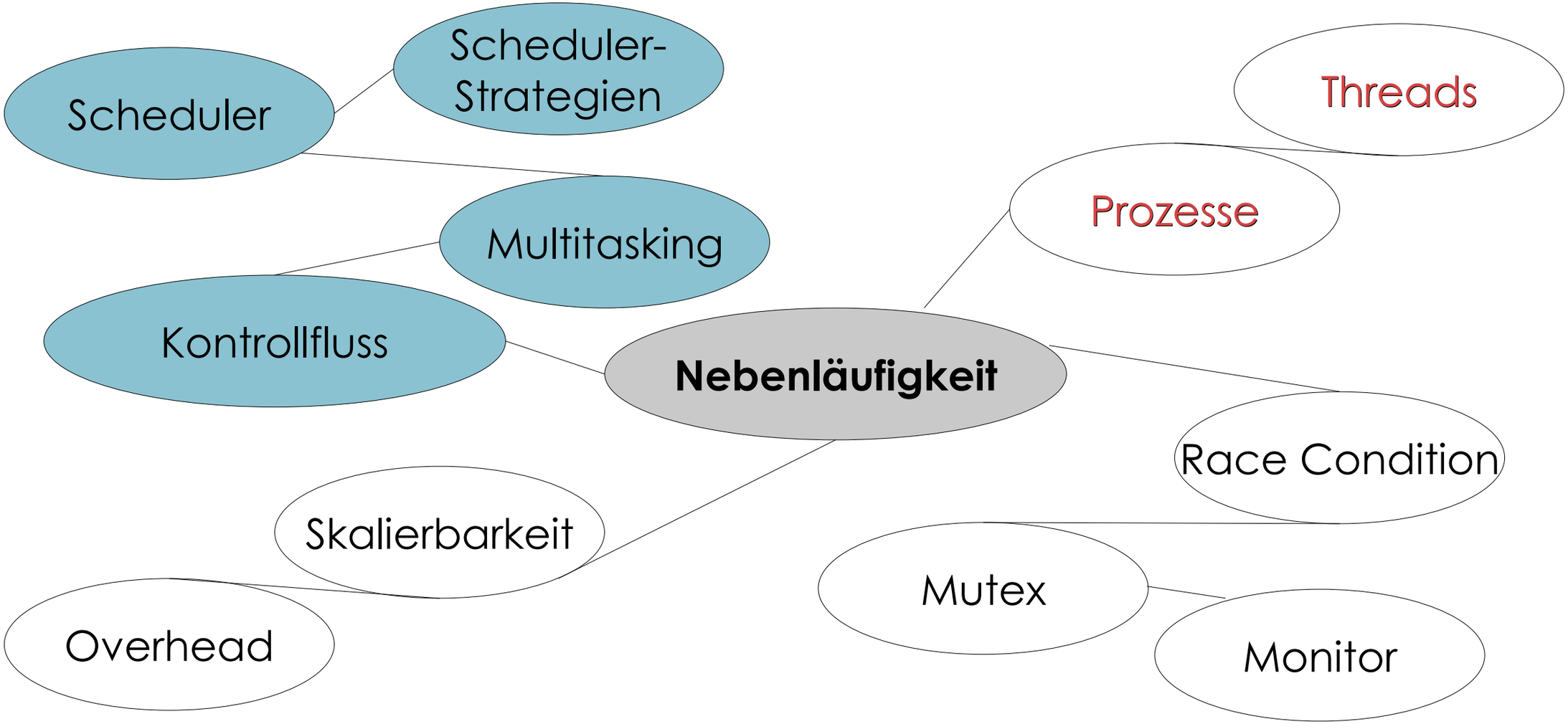
1) FIRST COME FIRST SERVE



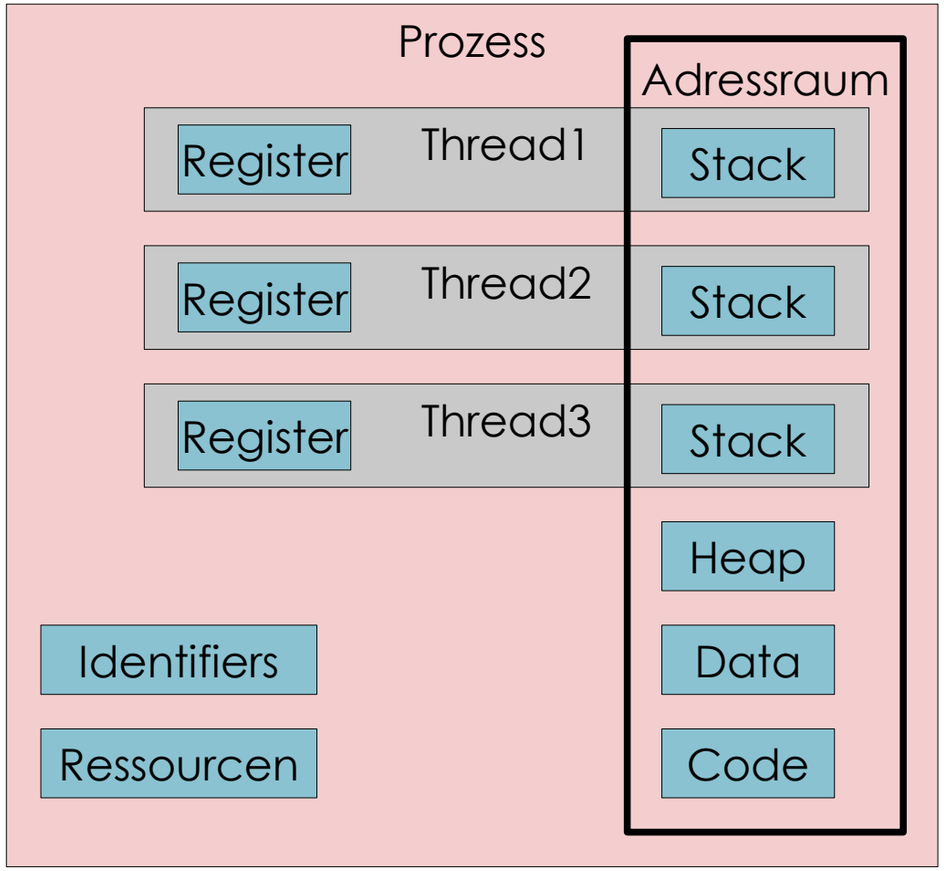
3) SHORTEST-REMAINING-TIME



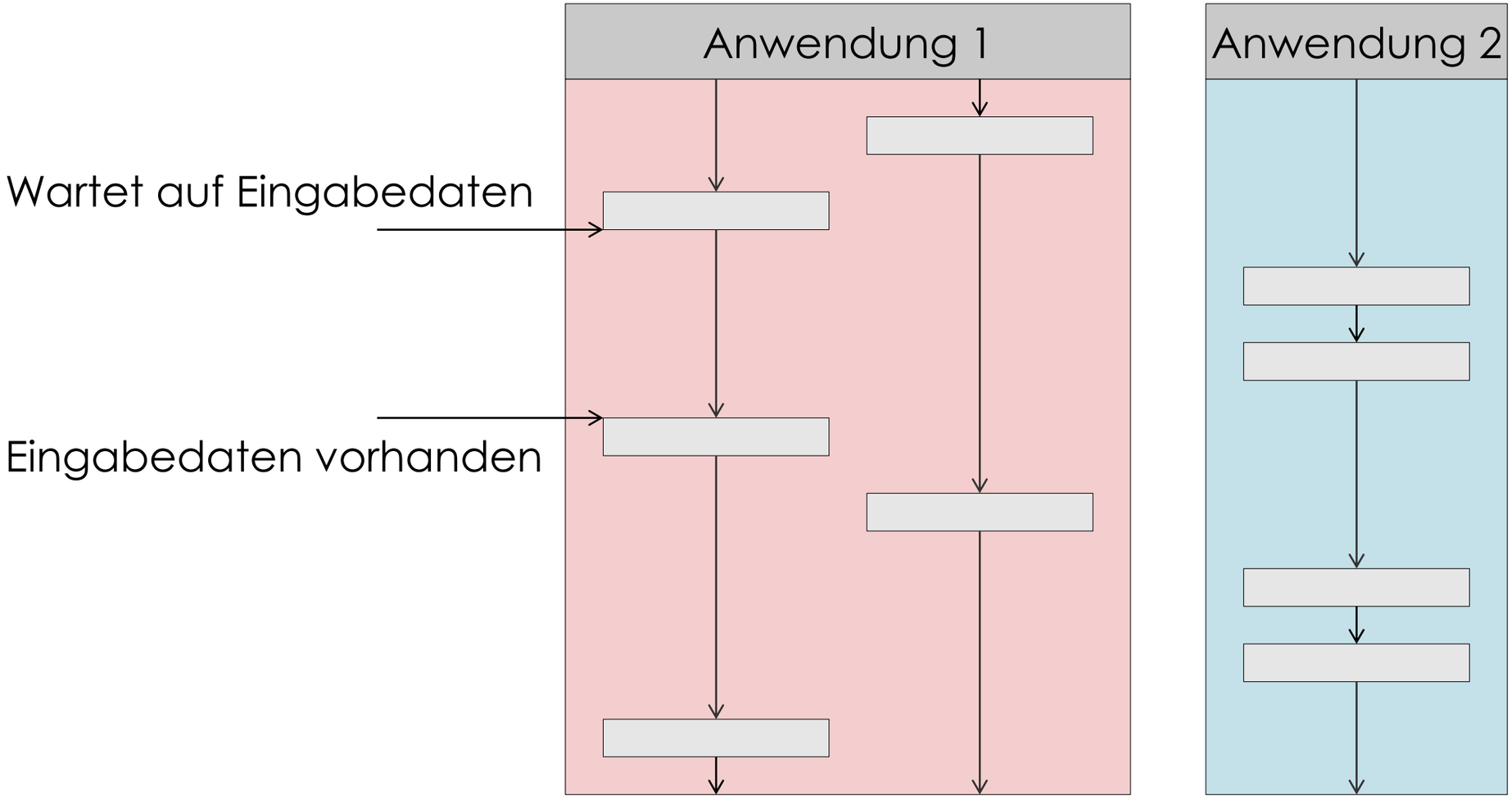
KONTEXT DER NEBENLÄUFIGKEIT



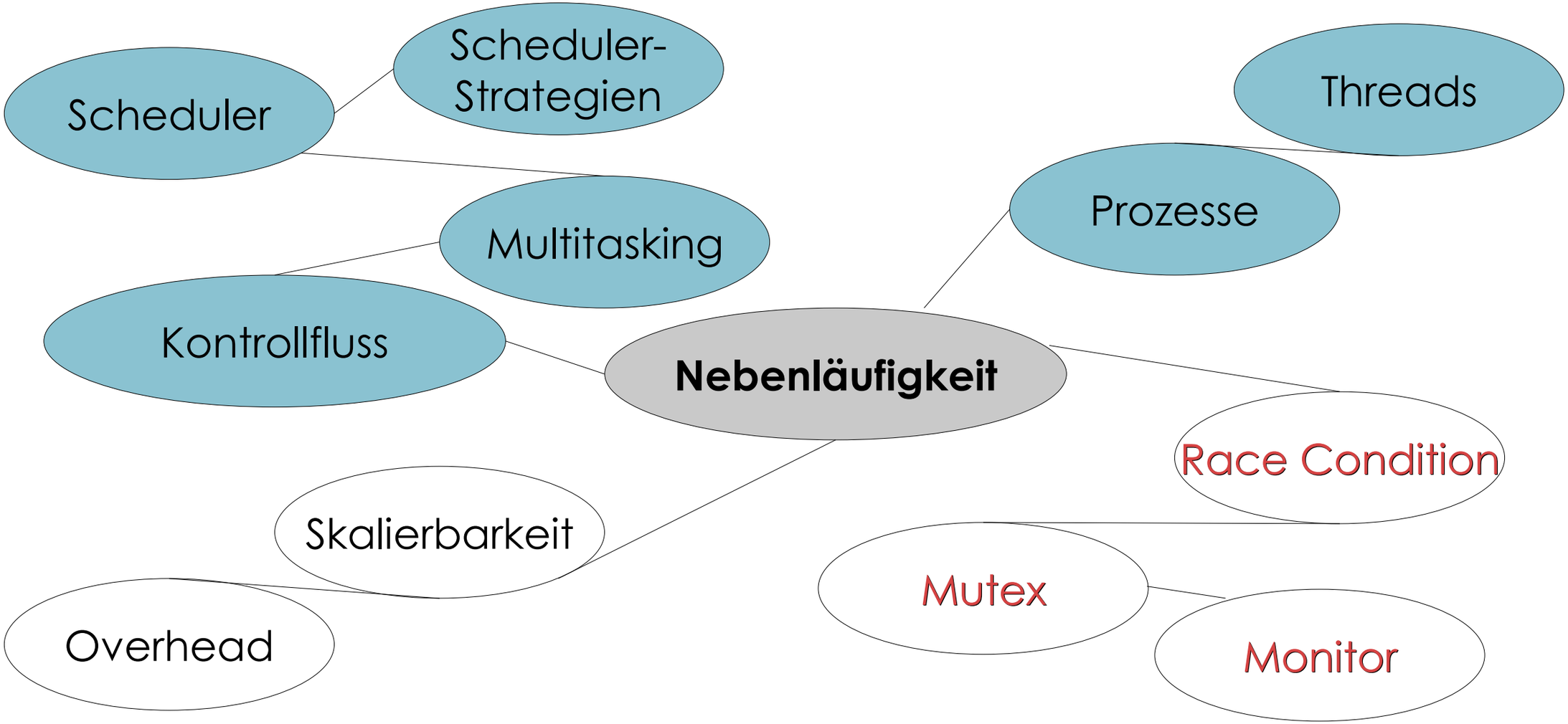
THREADS IM PROZESS



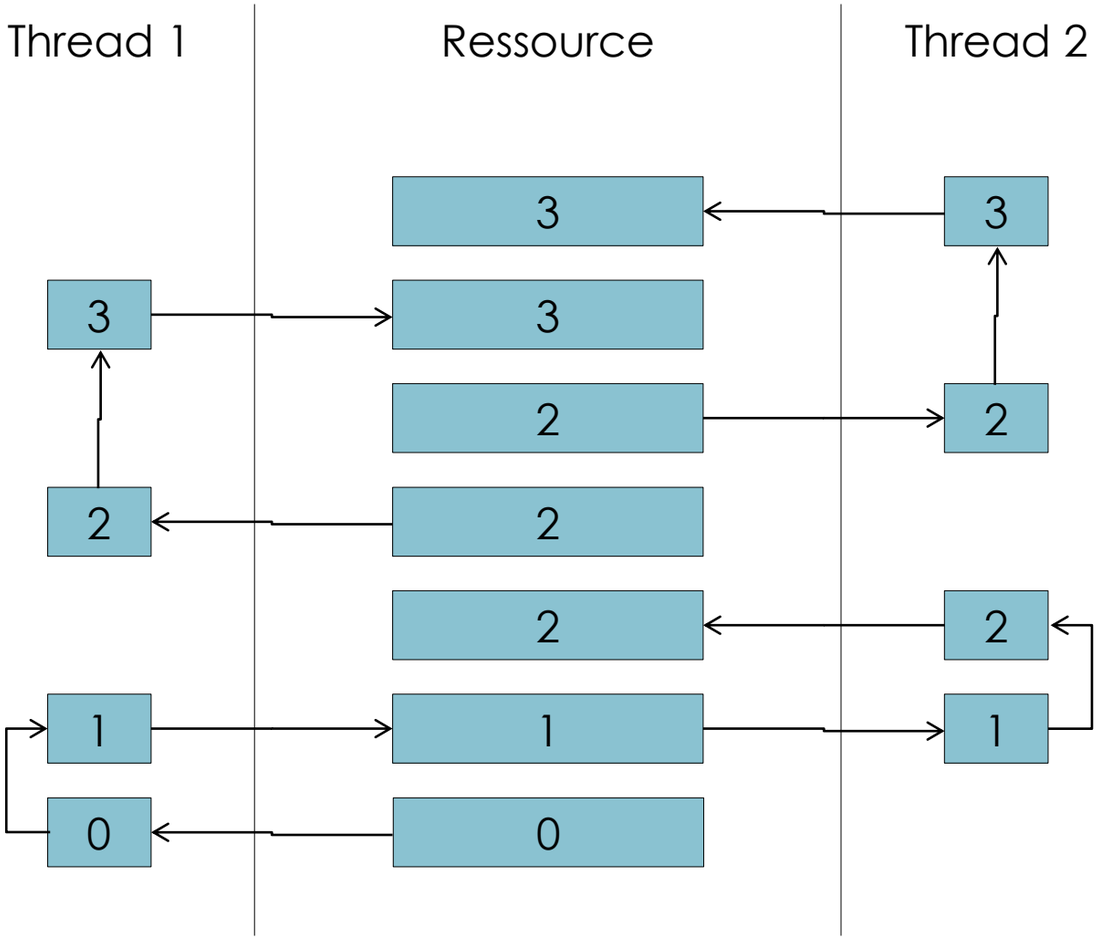
PROZESSE UND *THREADS*



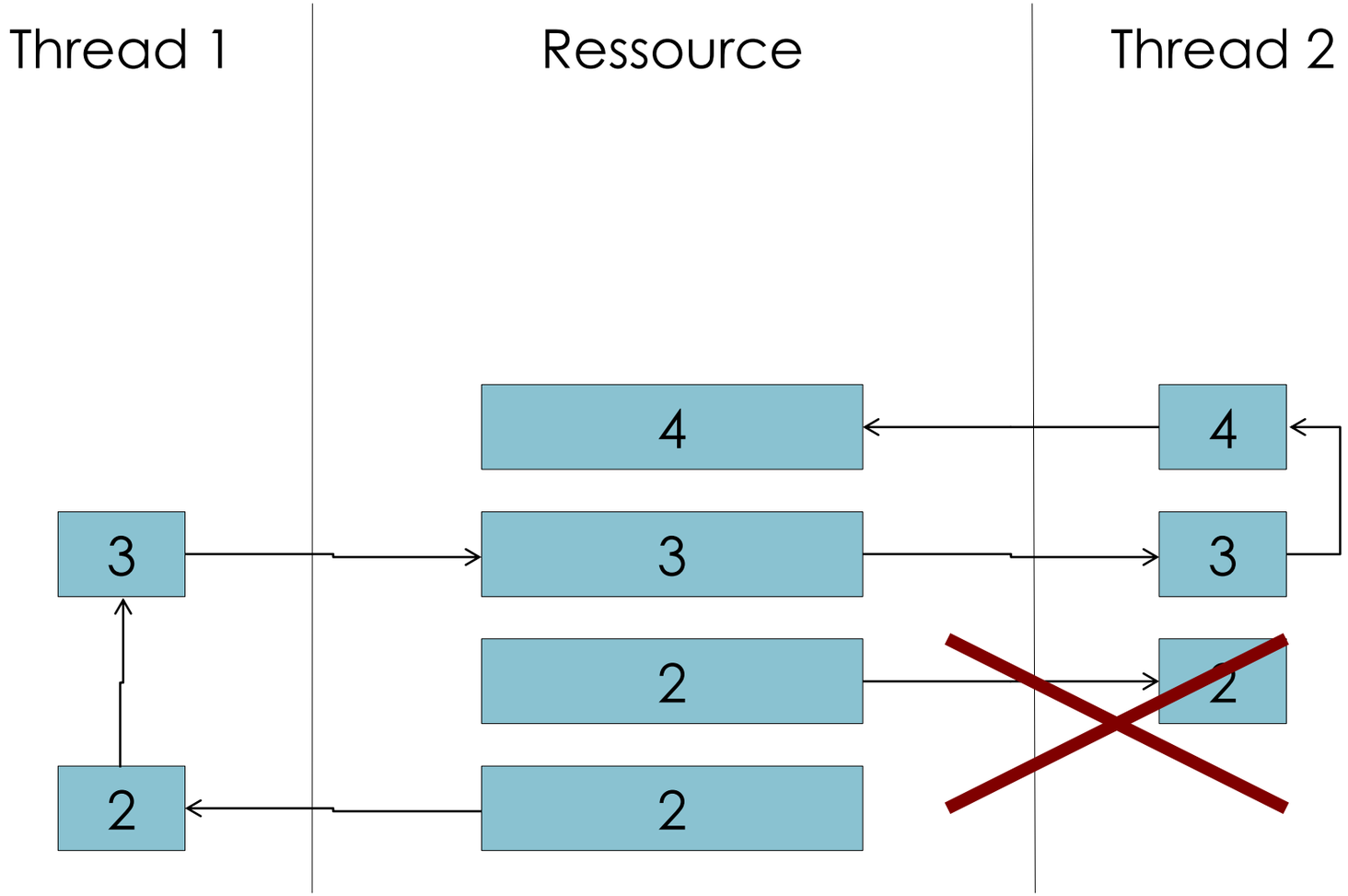
KONTEXT DER NEBENLÄUFIGKEIT



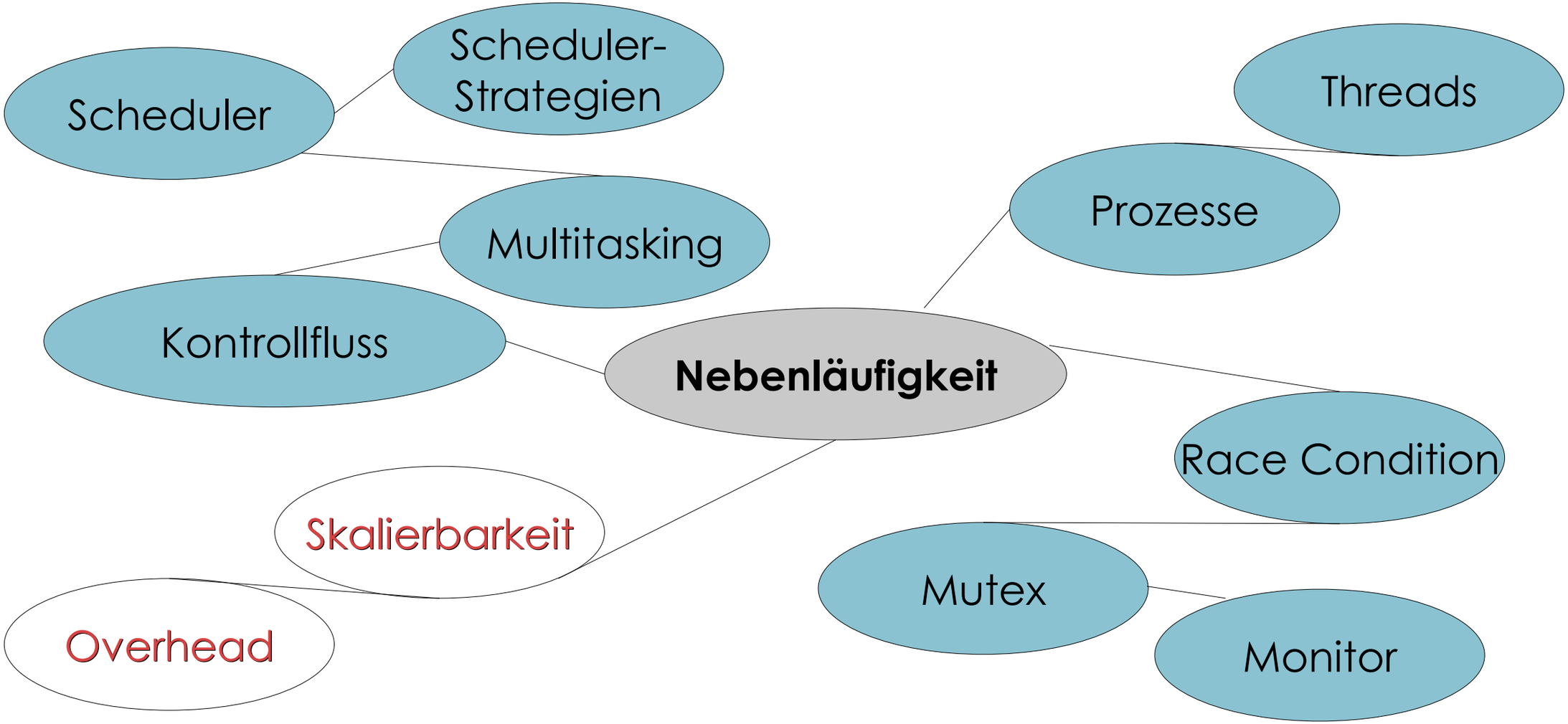
BEISPIEL RACE CONDITION



BEISPIEL MUTEX (ENGL. MUTUAL EXCLUSION)



KONTEXT DER NEBENLÄUFIGKEIT



ÜBERSICHT

- Serielle und parallele Verarbeitung
- Kontext der Nebenläufigkeit
- Probleme nebenläufiger Systeme
- Theoretische Modelle zur Nebenläufigkeit

PROBLEME NEBENLÄUFIGER ZUGRIFFE

Verlorene Aktualisierungen (engl. *lost update*)

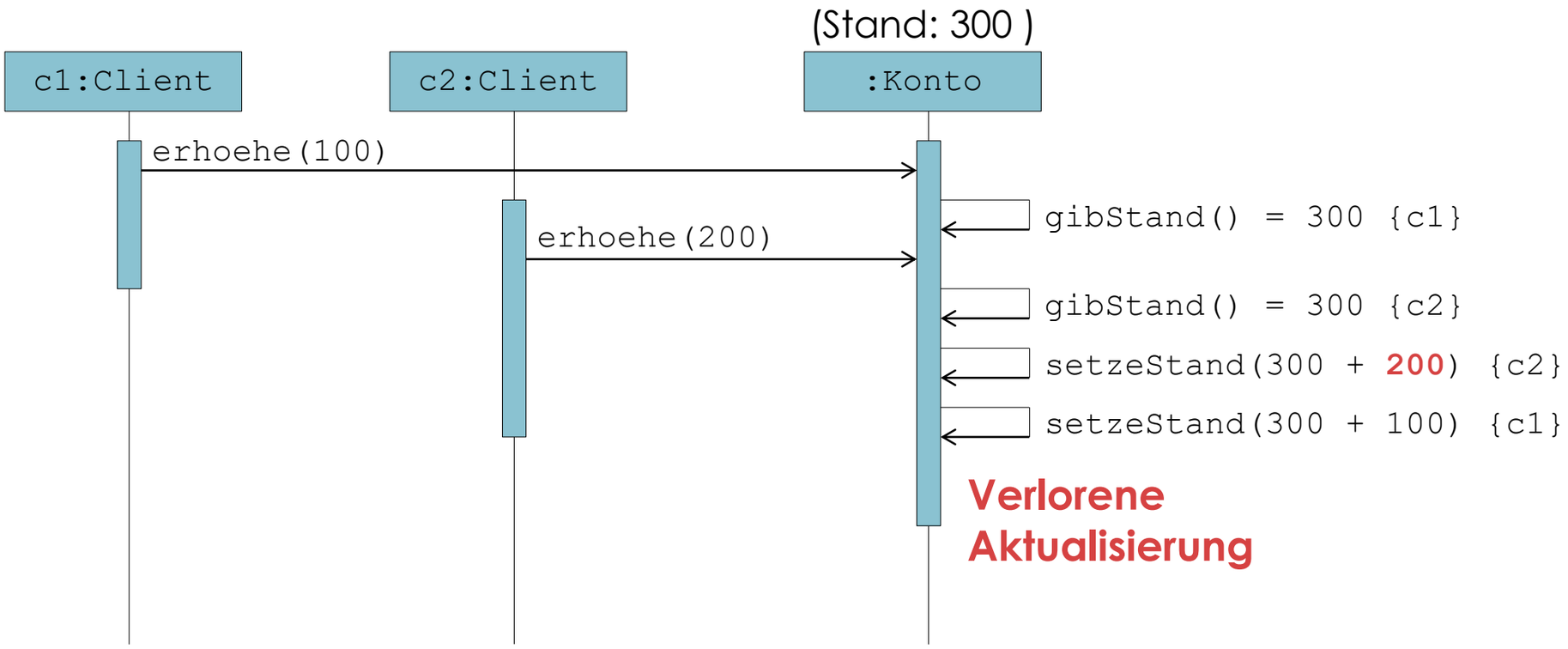
- Fehler bei parallelen Schreibzugriffen auf geteilte Information
- Zwei Kontrollflüsse verändern dieselbe Information
  - Erster ändert Information, ist aber noch nicht beendet
  - Zweiter überschreibt die Änderung
  - Änderung der ersten Transaktion geht verloren
- Eine Form der Race Condition

BEISPIEL: KONTO

Konto
- name - stand
+ erhoehe(betrag) + verringere(betrag) + gibStand() - setzeStand(betrag)

```
void erhoehe(int betrag) {
    setzeStand(gibStand() + betrag);
}
```

VERLORENE AKTUALISIERUNG



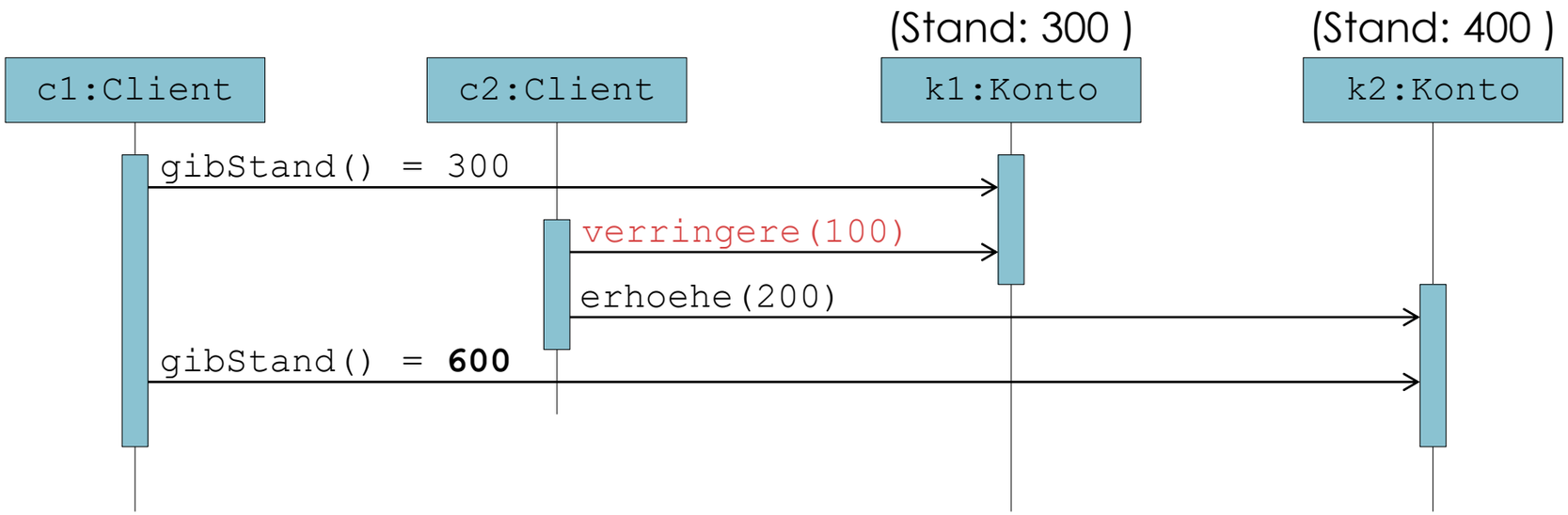

```
void erhoehe(int betrag) {
    setzeStand(gibStand() + betrag);
}
```

PROBLEME NEBENLÄUFIGER ZUGRIFFE

Inkonsistente Analyse (*inconsistent analysis*)

- Kontrollfluss erstellt Analyse aus mehreren Daten
- Daten werden während Analyse von zweitem Kontrollfluss verändert
- Erster Kontrollfluss registriert dies nicht
- Ergebnis ist falsche / nicht aktuelle Analyse

INKONSISTENTE ANALYSE



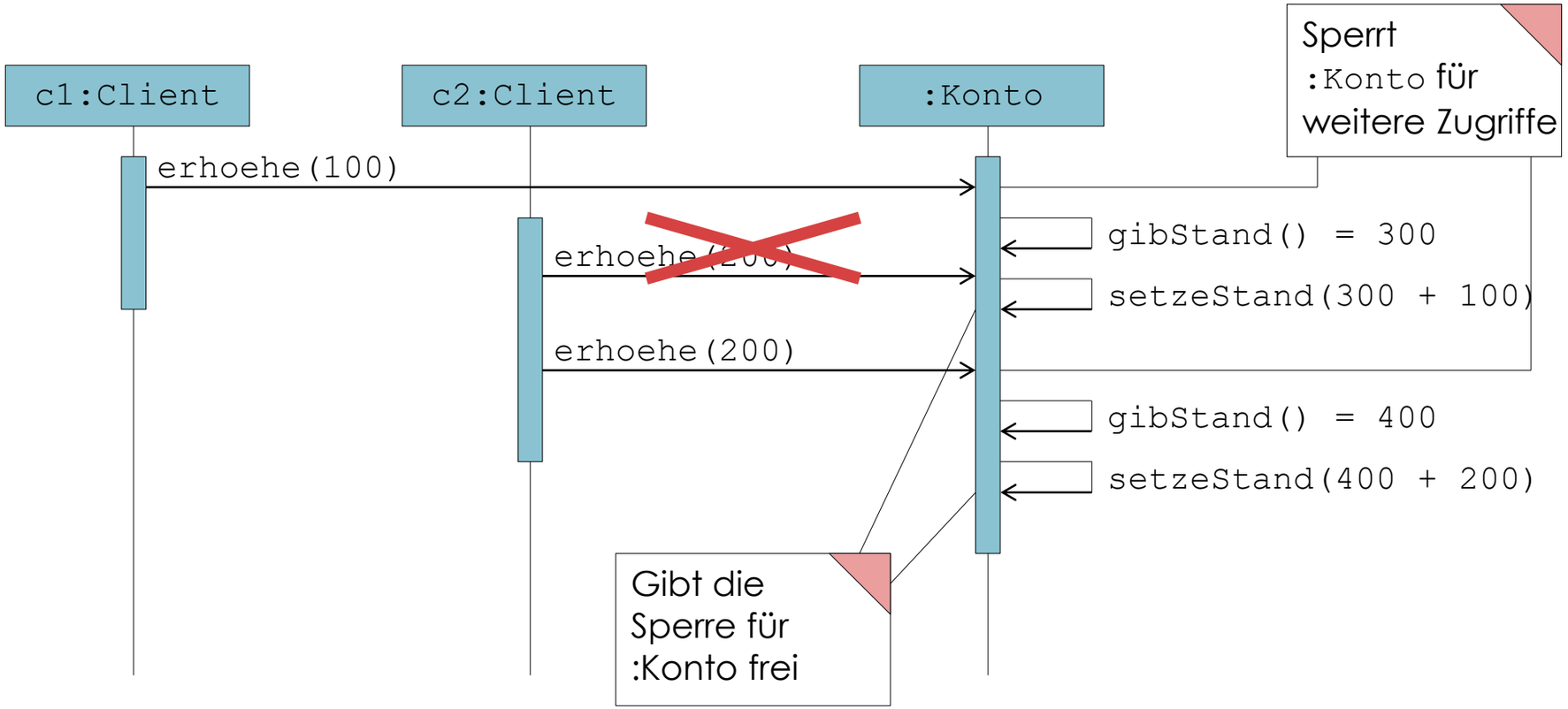
Folge: Inkonsistentes Analyseergebnis bei Summenbildung

```
Konto k1;
Konto k2;
...
int summe = 0;
summe = k1.gibStand() + k2.gibStand();
```

LÖSUNG MUTEX

- Mutex zur Synchronisation mehrerer Kontrollflüsse bereits bekannt
- Kontrollstruktur zum Schutz eines **kritischen Bereichs**
- Führt zur Einschränkung der nebenläufigen Verarbeitung
- Kritischen Bereich so klein wie möglich halten
- Andere Kontrollflüsse müssen auf Freigabe des Bereichs warten
  - Aktives Warten (zyklische Abfrage)
  - Passives Warten („Schlafen“ bis er benachrichtigt wird)

BEISPIEL: NEBENLÄUFIGER ZUGRIFF MIT MUTEX




```
void erhoehe(int betrag) {
    setzeStand(gibStand() + betrag);
}
```

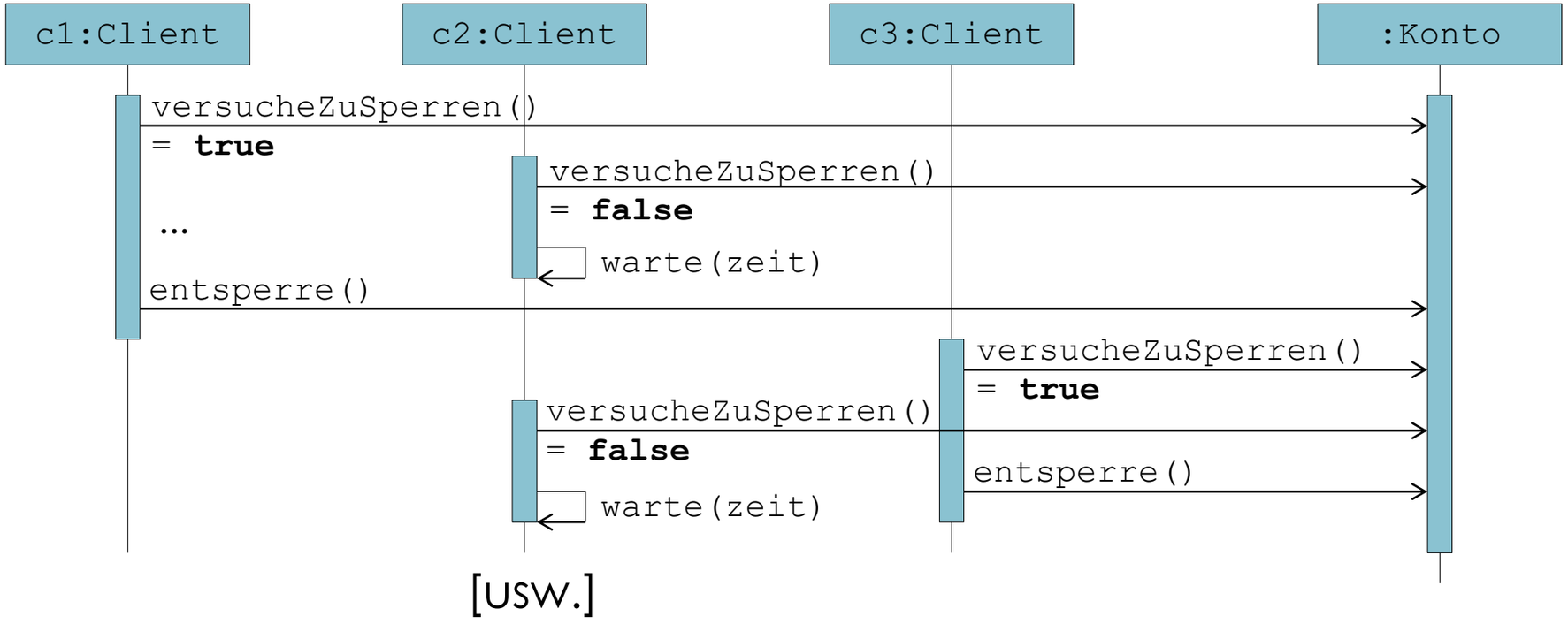
Mutex

PROBLEME DURCH MUTEX

Verhungernder Kontrollfluss

- Kontinuierlich warten mehrere Kontrollflüsse auf Mutex-Vergabe
- Beim Warten auf einem Mutex wird Kontrollfluss deaktiviert
- Mutex-Vergabe findet nicht deterministisch statt
- Zum Beispiel nur an aktive Kontrollflüsse oder rein zufällig
- Im Extremfall kann so ein Kontrollfluss nie zur Ausführung kommen
- Durch Einsatz von Scheduling-Algorithmen recht unwahrscheinlich

VERHUNGERNDER KONTROLLFLUSS

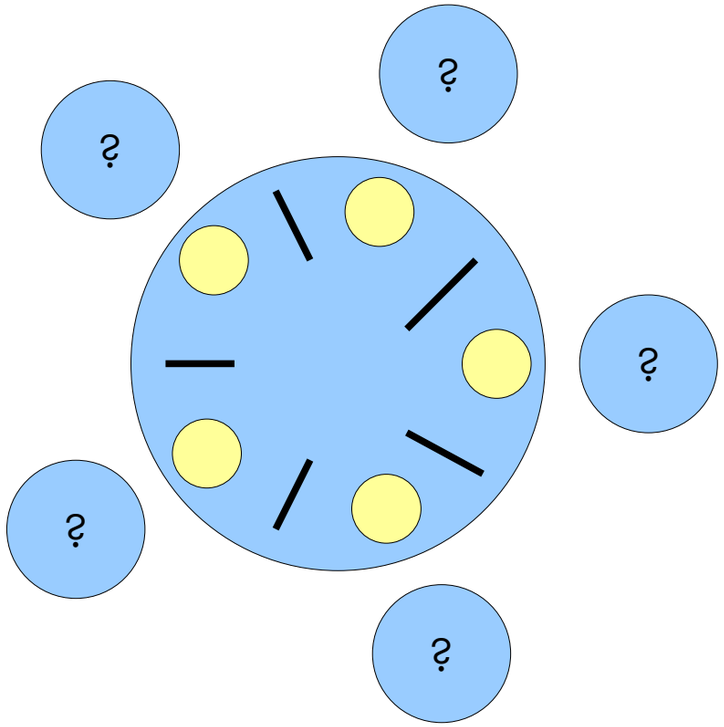


PROBLEME DURCH MUTEX

Verklemmungen (engl. *deadlocks*)

- Das gegenseitige Warten auf die Freigabe einer benötigten Ressource, ohne jedoch die eigene zuerst freizugeben
- Voraussetzungen
  - Mindestens zwei Kontrollflüsse sind im System aktiv,
  - Diese greifen auf dieselben (mindestens zwei) Ressourcen zu
  - Ressourcen werden durch *Mutex* synchronisiert
  - Kontrollflüsse sichern sich jeweils einen Teil der Ressourcen
  - Benötigen aber noch die weiteren Ressourcen zur Ausführung
  - Diese sind jedoch bereits durch andere Kontrollflüsse reserviert
  - Freigabe der Ressourcen erfolgt ausschließlich kooperativ(?)

PHILOSOPHENPROBLEM (VERKLEMMUNG)



- Nachdenken
- Hunger
  - Linkes Stäbchen nehmen
  - Rechtes Stäbchen nehmen
- Essen
- Beide Stäbchen zurücklegen

Backup

PHILOSOPHENPROBLEM (VERKLEMMUNG)

- Fünf Philosophen sitzen am Tisch und denken über philosophische Probleme nebenläufiger Systeme nach
- Wird einer hungrig so benötigt er zwei Stäbchen zum Essen
- Er nimmt zunächst das linke, dann das rechte Stäbchen
- Sobald er satt ist, legt er beide Stäbchen wieder zurück und denkt wieder über philosophische Probleme nebenläufiger Systeme nach
- Ist ein Stäbchen nicht an seinem Platz so wartet der Philosoph bis es wieder verfügbar ist
- Hat er bereits ein Stäbchen in der Hand, legt er es nicht zurück
- Parallele Kontrollflüsse (Philosophen) sind auf gemeinsame Ressourcen (Stäbchen) angewiesen

Backup

PHILOSOPHENPROBLEM (VERKLEMMUNG)

- Funktioniert sehr gut, sofern nur einzelne Philosophen hungrig sind
- Falls alle Philosophen zur gleichen Zeit hungrig sind wird es allerdings problematisch
  - Alle ergreifen zuerst das linke Stäbchen
  - Nehmen dem links sitzenden Kollegen sein rechtes Stäbchen weg
  - Alle warten anschließend darauf, dass das rechte Stäbchen auftaucht
  - Da keiner der Philosophen das linke Stäbchen wieder zurücklegt, verhungern alle Philosophen
- Vergebliches Warten auf Ereignis das aufgrund der Blockade nie eintrifft - **Verklemmung/Deadlock**

VERHINDERN VON VERKLEMMUNGEN

- Ein Kontrollfluss gibt bereits gesicherte Ressourcen frei
  - Wenn er einen benötigten *Mutex* nicht erhält
  - *Mutex* kann auch von übergeordneter Stelle entzogen werden (Monitor)
  - Aufwändig, speziell wenn viele Ressourcen gesperrt werden müssen
- Jeder Kontrollfluss gibt vorher an, welche Ressourcen er benötigt
  - Wenn diese alle zum selben Zeitpunkt frei sind, bekommt der Kontrollfluss für alle Ressourcen den *Mutex*
  - Kann sehr lange dauern – Verhungernder Kontrollfluss
- Ressourcen werden durchnummeriert und nur in aufsteigender Reihenfolge vergeben
- Auflösung des *Mutex* – benötigte Ressourcen für alle Prozesse wieder zugänglich machen

BESEITIGUNG VON VERKLEMMUNGEN

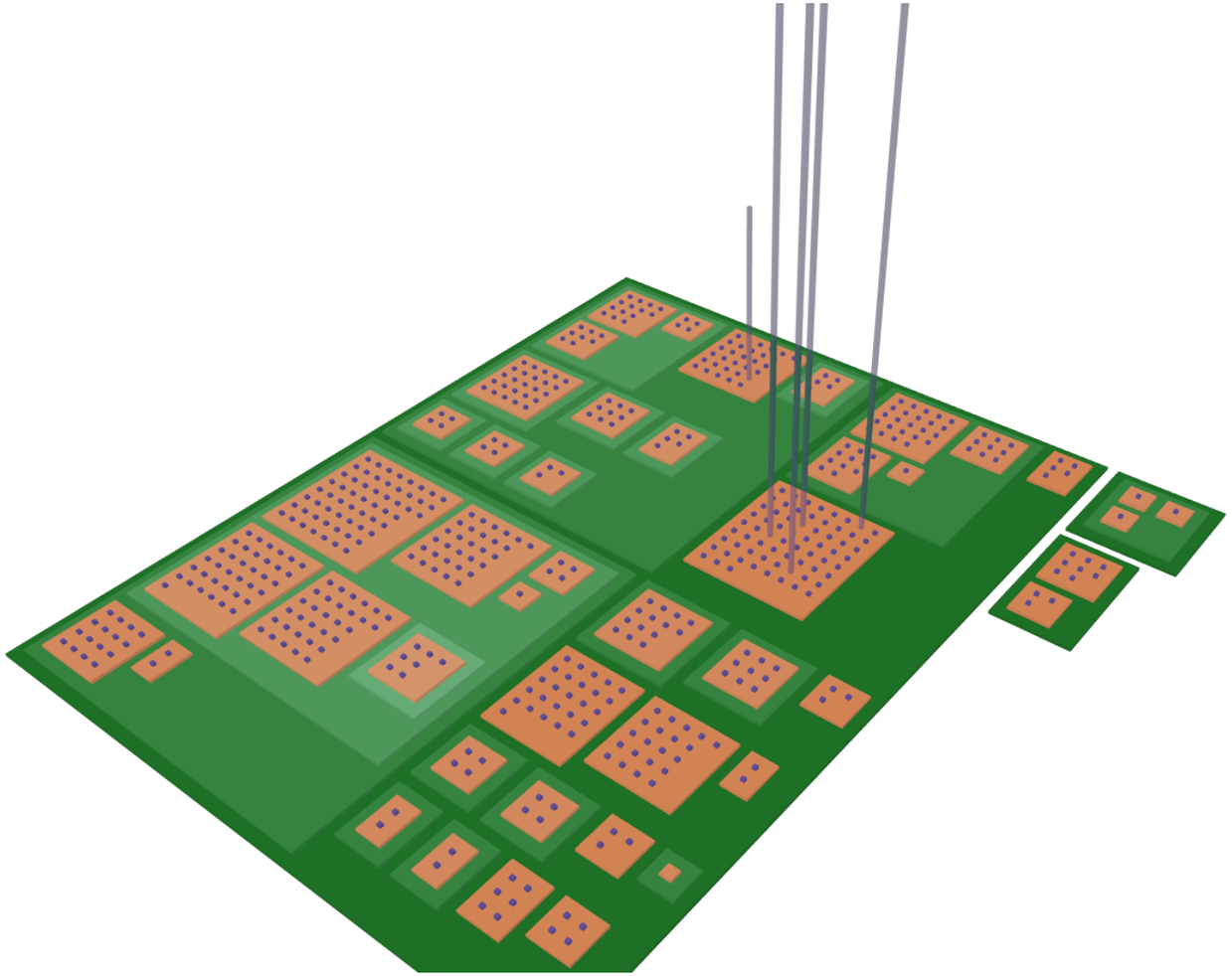
- Prozessabbruch
  - Einfachste Variante
  - Gute Auswahl des Prozesses wichtig – schlecht automatisierbar
  - Datenverlust nicht vermeidbar nur einschränkbar
- Präemption
  - Prozess unterbrechen und erst später fortsetzen
  - Zwischenzeitlich arbeiten die blockierten Prozesse ihre Aufgaben ab
- Rollback
  - Sicherungen jedes Prozesses vorher notwendig
  - Im Falle einer Verklemmung wird ein Rollback des betroffenen Prozesses auf gesicherten Stand vorgenommen
  - „Neustart“ des Prozesses sobald Ressourcen wieder frei sind

ÜBERSICHT

- Serielle und parallele Verarbeitung
- Kontext der Nebenläufigkeit
- Probleme nebenläufiger Systeme
- Theoretische Modelle zur Nebenläufigkeit

UNIVERSITÄT
LEIPZIG

VIELEN DANK!

Pascal Kovacs
Institut für Wirtschaftsinformatik

pkovacs@uni-leipzig.de
https://www.wifa.uni-leipzig.de/iwi/